

Assignment — Day 1 & Day 2

Practical Cyber Security (v2) · Week 1

Everything here also lives in `week1/student-pack.md`. This handout is just Day 1 + Day 2 pulled out on their own, so you have something to give students without previewing Day 3–5.

Day 1 homework — due start of Day 2

1. Set up your course repo

1. Create a free account at github.com **or** gitlab.com (turn on 2-factor auth).
2. Create a **private** repository named `cybersec-course`.
3. Install Git: Windows → <https://git-scm.com/download/win> · Linux → `sudo apt install git` · macOS → `xcode-select --install`.
4. `git clone <your repo URL>` then `cd cybersec-course`.
5. Create `README.md` (see item 2 below), then: `git add . && git commit -m "day 1" && git push`
6. Structure to build over the course: `README.md`, `day01/`, `day02/`, ... one folder per day.

[Stuck? Bring your laptop to the start of Day 2 — we'll fix it in the first 5 minutes.](#)

2. Write `README.md` — two short paragraphs: **why** you're taking this course, and the sentence *"I will only test systems I am authorized to test."* Start a "Today I learned" list.

3. Read and sign the Authorization Pledge below. [Bring the signed copy to Day 2.](#)

AUTHORIZATION PLEDGE

I understand that accessing or testing computer systems without authorization is unlawful under the Information Technology Act, 2000, regardless of intent or outcome. I will only test systems I am authorized to test — my own isolated lab, or systems for which I hold explicit written permission or a valid bug-bounty scope. I will report vulnerabilities responsibly and will not use anything taught in this course to cause harm.

Name: _____ Roll no: _____ Signature: _____
Date: _____

Day 2 homework — due start of Day 3

1. Draw the **source code** → **executable** → **running process** pipeline yourself. Label where the **CPU**, **RAM**, and **disk** each come in.
2. One paragraph: **what is the difference between a program and a process?**
3. Commit both to your repo; update your README's "today I learned" list.

Reminder of the model from class: you write source code → a compiler turns it into machine code (an executable file on disk) → the OS loads it into RAM and points the CPU at it → it's now a running process, identified by a PID.

BONUS (OPTIONAL, WINDOWS + ADMIN RIGHTS ONLY) — PROCESS TRACKING IN EVENT VIEWER

Not required, and not gradable if you can't do it — skip it if you're on Linux/macOS or don't have admin rights on your machine.

Worked example (Notepad) — for reference only, don't submit this one:

1. Command Prompt / PowerShell, opened as **Administrator** (right-click → "Run as administrator"): `auditpol /set /subcategory:"Process Creation" /success:enable` → should print "The command was successfully executed." ("Access is denied" means you're not elevated — reopen as admin.)
2. If Notepad is already open, **close it completely first** — you need a fresh launch *after* auditing was turned on, or there's nothing to find.

3. Open Notepad. Task Manager → Details → `notepad.exe` → PID **8420** (decimal).
4. Open Event Viewer: press Win, type `eventvwr`, Enter — accept the UAC prompt if it appears (admin is needed to read the Security log).
5. Windows Logs → **Security** → right-click → **Filter Current Log...** → type `4688` in the Event ID box → OK.
6. Newest events are at the top by default — open the one right after you launched Notepad.
7. Under **Process Information**: “New Process Name” = `notepad.exe`, “New Process ID” = `0x20E4` (hex). **Read “New Process ID,” not “Creator Process ID”** (Creator is the parent that launched Notepad, e.g. `explorer.exe` — not Notepad itself).
8. Convert `0x20E4` to decimal: Windows Calculator → menu → **Programmer** mode → type `20E4` with Hex selected → switch to Dec → reads **8420**, matching Task Manager.

Your task — pick a DIFFERENT program that runs as a single process (Calculator, Paint, Notepad++, VS Code — anything except Notepad. Avoid Chrome/Edge/most browsers: they launch several processes at once, making it hard to tell which PID is “the” one):

1. Close it first if already open, then open it fresh. Task Manager → Details tab → note its **PID** (decimal).
2. Find its matching Event ID 4688 entry in Event Viewer (same steps as above).
3. Confirm “New Process Name” matches, and convert “New Process ID” (hex) to decimal — it should equal the PID from Task Manager.
4. Screenshot the event, and write one sentence: what does this prove about how the OS keeps a record of every process it creates?

If no 4688 events show up at all: run `auditpol /get /subcategory:"Process Creation"` to confirm “Success” is enabled. If it is, but the log stays empty, you’re likely on a school/managed laptop where Group Policy overrides this setting — try your own machine instead.

Bonus (optional, Windows) — explore and set up WSL

A real Linux kernel running next to your Windows kernel, on the same machine — the live version of today’s “kernel” idea. It also lets you actually run the Linux commands used all course (`htop`, `ip a`, `dig`, ...) yourself instead of only reading the Windows equivalent.

1. Open PowerShell **as Administrator** → `wsl --install`. Restart if prompted.
2. After restart, Ubuntu launches automatically and finishes installing — create a UNIX username and password (separate from your Windows login; the typed password won’t show characters, that’s normal).
3. Check it worked: on Windows, `wsl --status`; inside Ubuntu, `uname -a` (should mention `microsoft-standard-WSL2`).
4. Inside Ubuntu: `sudo apt update && sudo apt install -y htop`, then run `htop`. Compare it side by side with Task Manager — same idea (processes, PID, memory), a completely separate kernel managing it.
5. One sentence: is the Ubuntu you just opened a separate physical computer, a separate virtual machine, or something else?

(Not the same as the Kali attacker VM used later for Days 12–20 labs — WSL2 isn’t used there; see `LAB-SETUP.md`. This is just to get comfortable with a Linux shell early.)

Submission checklist

- ☐ Repo created, private, named `cybersec-course`
- ☐ `README.md` has: why-I’m-here paragraph, the authorization sentence, “today I learned” list
- ☐ Authorization Pledge signed and handed in (paper) or scanned into the repo
- ☐ Day 2 pipeline diagram committed (`day02/` folder)
- ☐ Day 2 program-vs-process paragraph committed
- ☐ Both days’ entries added to the “today I learned” list